Part 4 - Pygame Flappy Bird Tutorial

Contents

Part 4 - Pygame Flappy Bird Tutorial.....1
Full Game 1
Assignment Submission.....9
Assignment Submission.....10

Time required: 30 minutes

Full Game

Time to finish up the game.

1. Copy and paste the **flappy_bird_3.py** file. You will end up with a file named flappy_bird_1 copy.py
2. Rename it to **flappy_bird.py**
3. Modify the existing code

This is the complete project in case you are missing something.

```python
"""
    Name: flappy_bird.py
    Author:
    Date:
    Purpose: Flappy Bird Clone in OOP
"""

# pip install pygame
import pygame
import random
import time


class FlappyBird:
    def __init__(self):
        # Setup color constant for sky
        self.SKY_BLUE = (135, 206, 235)
        # Initialize pygame engine
        pygame.init()

        # Surface width and height constants
        self.SURFACE_WIDTH = 800
        self.SURFACE_HEIGHT = 500
        # Set screen width and height as a tuple
        self.SURFACE = pygame.display.set_mode(
            (self.SURFACE_WIDTH, self.SURFACE_HEIGHT)
        )
        # Define the clock to keep the game running at a set speed
        self.CLOCK = pygame.time.Clock()

        self.init_bird()
        self.init_pipes()

        self.score = 0
        self.game_over = False
        self.pass_pipe = False
        self.score_font = pygame.font.SysFont("arialblack", 18)
```

```python
#------------------------- INIT FLAPPY BIRD ----------------------------------------#
    def init_bird(self):
        """
            Load bird images, get rect, set initial position
        """
        # Load flappy bird program png icon
        self.bird_ico = pygame.image.load(
            "./images/flappy_bird_ico.png").convert_alpha()
        pygame.display.set_icon(self.bird_ico)
        pygame.display.set_caption("Flappy Bird")

        # Load the bird image and get rectangle
        self.bird = pygame.image.load(
            "./images/flappy_bird.png").convert_alpha()
        # Get rectangle around bird for easier manipulation
        self.bird_rect = self.bird.get_rect()

        # Set bird to initial position
        self.bird_rect.move_ip(
            200,
            self.SURFACE_HEIGHT / 2
        )
```

```python
#------------------------------ INIT PIPES ----------------------------------------#
    def init_pipes(self):
        """
            Load pipe images, get rect, set initial positions
        """
        # Set the gap between pipes
        self.pipe_gap_size = self.bird_rect.height * 5
        # How many pixels at a time the pipes move
        self.pipe_move = 4
        # Load pipe images
        self.pipe_lower = pygame.image.load(
            "./images/pipe.png").convert_alpha()
        # Rotate upper image 180 degrees
        self.pipe_upper = pygame.transform.rotate(
            pygame.image.load("./images/pipe.png").convert_alpha(), 180)

        # Get rectangles around images for easier manipulation
        self.pipe_lower_rect = self.pipe_lower.get_rect()
        self.pipe_upper_rect = self.pipe_upper.get_rect()

        # Set initial pipe location off screen to right
        self.pipe_upper_rect.left = self.SURFACE_WIDTH
        self.pipe_lower_rect.left = self.SURFACE_WIDTH

        # Initial placement of pipes vertically
        self.pipe_upper_rect.bottom = random.randint(
            50,
            self.SURFACE_HEIGHT / 2
        )
        # Set lower pipe vertical location
        self.pipe_lower_rect.top = self.pipe_upper_rect.bottom + self.pipe_gap_size
        # Set score counted to false
        self.score_counted = False
```

```python
#----------------------------- RESET PIPES ----------------------------------------#
    def reset_pipes(self):
        """
            Reset pipes every time they leave the screen
        """
        # Pick a random height for the bottom of the top pipe
        self.pipe_upper_rect.bottom = random.randint(
            50,  # Set minimum random number to 50
            self.SURFACE_HEIGHT / 2  # Set maximum to half the surface height
        )
        # Set lower pipe top to upper pipe bottom plus pipe gap
        self.pipe_lower_rect.top = self.pipe_upper_rect.bottom + self.pipe_gap_size
        # Set initial X off screen to right
        self.pipe_upper_rect.left = self.SURFACE_WIDTH
        self.pipe_lower_rect.left = self.SURFACE_WIDTH
        # New set of pipes, reset score counter
        self.score_counted = False

#----------------------------- DISPLAY SCORE --------------------------------------#
    def display_score(self):
        """
            Display the score on the screen
        """
        # Create text image for score display
        text = self.score_font.render(f"Score: {self.score}", True, "white")
        self.SURFACE.blit(
            text,   # Image to display
            [3, 3]  # x , y to display the image
        )

#------------------------- CREATE TEXT OBJECT -------------------------------------#
    def make_text_objects(self, text, font):
        """
            Create a text surface and rect to display text
        """
        text_surface = font.render(text, True, "white")
        return text_surface, text_surface.get_rect()
```

```python
#-------------------------- CONTINUE OR EXIT -------------------------------------#
    def continue_or_exit(self):
        """
            Decide whether to continue or exit the game
        """
        while True:
            # Get only specific events
            for event in pygame.event.get([
                pygame.KEYDOWN,
                pygame.KEYUP,
                pygame.QUIT
            ]):

                # If the user closes the program window
                if event.type == pygame.QUIT:
                    pygame.quit()
                    quit()

                # If the user presses a key
                elif event.type == pygame.KEYDOWN:
                    # If the users press x, exit
                    if event.key == pygame.K_x:
                        pygame.quit()
                        quit()
                    # If the user presses c, reset the game
                    elif event.key == pygame.K_c:
                        main()
```

```python
#------------------------- DISPLAY GAME OVER -------------------------------------#
    def display_game_over(self):
        """
            Display game over on top of the stopped game
        """
        # Create System font objects
        small_text = pygame.font.SysFont("arialblack", 20)
        large_text = pygame.font.SysFont("arialblack", 130)

        # Create Game Over title text
        title_text_surface, title_text_rect = self.make_text_objects(
            "Game Over",
            large_text
        )
        # Set location of the center of the text
        title_text_rect.center = self.SURFACE_WIDTH / 2, self.SURFACE_HEIGHT / 2
        self.SURFACE.blit(
            title_text_surface,
            title_text_rect
        )

        # Create continue game text
        type_text_surface, type_text_rect = self.make_text_objects(
            "Press (C) to continue",
            small_text
        )
        # Set location of the center of the text
        type_text_rect.center = self.SURFACE_WIDTH / \
            2, ((self.SURFACE_HEIGHT / 2) + 100)
        self.SURFACE.blit(
            type_text_surface,
            type_text_rect
        )

        # Create x to exit text
        type_text_surface, type_text_rect = self.make_text_objects(
            "Press (X) to exit",
            small_text
        )
        type_text_rect.center = self.SURFACE_WIDTH / \
            2, ((self.SURFACE_HEIGHT / 2) + 150)
        self.SURFACE.blit(
            type_text_surface,
            type_text_rect
        )

        pygame.display.update()
        time.sleep(1)

        self.continue_or_exit()
```

```python
#------------------------------- GAME LOOP ----------------------------------------#
    def game_loop(self):
        """
            Infinite game loop
        """
        while not self.game_over:
            # Simulate gravity by moving the bird down unless the UP key is pressed
            # Reset gravity to 3 each time through the loop
            gravity = 3

            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    self.game_over = True

            # Get list of keys being pressed
            key_input = pygame.key.get_pressed()
            # If up cursor pressed, move up 5 pixels
            if key_input[pygame.K_UP]:
                gravity -= 5

        #------------------------- START DRAWING -------------------------------#
            # Draw everything on the backbuffer first
            # Fill the display surface with blue
            self.SURFACE.fill(self.SKY_BLUE)

            # Draw all images to the backbuffer
            self.SURFACE.blit(
                self.bird,      # Source image
                self.bird_rect  # Destination location of image
            )
            self.SURFACE.blit(
                self.pipe_lower,
                self.pipe_lower_rect
            )
            self.SURFACE.blit(
                self.pipe_upper,
                self.pipe_upper_rect
            )

            self.display_score()

        #------------------------- INCREASE DIFFICULTY -------------------------#
            # Adding difficulty relative to score
            # Increase the speed and decrease the gap of blocks
            if 5 <= self.score < 10:
                self.pipe_move = 6
                self.pipe_gap_size = self.bird_rect.height * 4

            elif 10 <= self.score < 20:
                self.pipe_move = 7
                self.pipe_gap_size = self.bird_rect.height * 3.5
```

```
265         #------------------- COLLISION DETECTION ---------------------------------#
266             # If the bird hits the top or bottom of screen, game over
267             if self.bird_rect.bottom > self.SURFACE_HEIGHT\
268                     or self.bird_rect.top < 0:
269                 self.display_game_over()
270
271             # The bird is between the pipes
272             if self.bird_rect.left > self.pipe_upper_rect.left\
273                     and self.bird_rect.right < self.pipe_upper_rect.right:
274                 # If the bird runs into a pipe while passing the pipes, game over
275                 if self.bird_rect.top < self.pipe_upper_rect.bottom\
276                         or self.bird_rect.bottom > self.pipe_lower_rect.top:
277                     self.display_game_over()
278
279         #------------------------------- SCORING --------------------------------#
280             # If the bird makes it past the pipes successfully, increase score
281             if self.bird_rect.left > self.pipe_upper_rect.right\
282                     and not self.score_counted:
283                 # Increase score
284                 self.score += 1
285                 # Track whether the current set of pipes have had a score
286                 self.score_counted = True
287
288         #------------------------ MOVE SPRITES ----------------------------------#
289             # Move the bird
290             self.bird_rect.y = self.bird_rect.y + gravity
291
292             # Move pipe images from right to left
293             self.pipe_upper_rect.left = self.pipe_upper_rect.left - self.pipe_move
294             self.pipe_lower_rect.left = self.pipe_lower_rect.left - self.pipe_move
295
296             # If the pipes are off the screen, reset them
297             if self.pipe_upper_rect.right < 0:
298                 self.reset_pipes()
299
300             # Update only the surfaces that need to be updated
301             pygame.display.update()
302             self.CLOCK.tick(60)
303
304
305 # Program entry point, main function
306 def main():
307     # Create flappy bird program object
308     flappy_bird = FlappyBird()
309     # Start infinite game loop
310     flappy_bird.game_loop()
311
312
313 # Start the program
314 main()
```

Example run:

A complete game!

There is always room for improvement.

1. Add more difficulty levels.
2. Keep track of the highest score.
3. Add music.
4. Etc.

Assignment Submission

Attach all program files to the assignment in BlackBoard.

Assignment Submission

Zip up the program files folder and submit in Blackboard.